Assignment 3 - Practice A (English / Russian)

Подготовительный вариант A

30 questions • Suggested time: 30 minutes • One correct choice per question.

30 вопросов • Рекомендуемое время: 30 минут • Один правильный ответ.

Read either language. Vertex and input edge IDs start at 0. The answer key and explanations follow after all questions; complete the questions before reading them.

Читайте удобный язык. Вершины и ID рёбер нумеруются с 0. Ответы с объяснениями идут после всех вопросов; сначала решите задания.

These examples practise the same skills as the graded quiz, but are not graded questions. Java fragments use basic arrays, loops and arithmetic.

Примеры проверяют те же навыки, что основной квиз, но не входят в него. Java-фрагменты используют простые массивы, циклы и арифметику.

1. DSU trace

n=8; apply unions [(0, 1), (1, 2), (0, 2), (3, 3)] to singleton components. Which list gives [components,largest_size] AFTER EACH operation?

n=8; выполните union [(0, 1), (1, 2), (0, 2), (3, 3)] на одиночных компонентах. Какой список даёт [components,largest_size] ПОСЛЕ КАЖДОЙ операции?

A. [[7, 2], [6, 3], [6, 3], [5, 4]]

B. [[2, 7], [3, 6], [3, 6], [3, 6]]

C. [[7, 2], [6, 3], [6, 3], [6, 3]]

D. [[7, 2], [6, 3], [5, 4], [4, 5]]

2. Path compression state

What is the WHOLE parent array after this code? Saving next before the update lets the loop visit the old path.

Каким станет ВЕСЬ массив parent? Сохранение next до изменения позволяет пройти старый путь.

```
int[] parent = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 12, 13, 15};
int root = 10, x = 14;
for (int u=x; u!=root; ) {
    int next=parent[u];
    parent[u]=root;
    u=next;
}
```

A. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10, 10, 15]

B. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 12, 10, 15]

C. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 12, 13, 15]

D. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 14, 10, 11, 12, 13, 15]

3. Size, rank and largest

Distinct roots a,b have sizes 24,12 and ranks 2,1; current largest=24. Compare two separate correct DSUs: union by size, and union by rank (on equal ranks attach b under a). Which NEW roots, rank update and largest are correct? Both keep size at the new root.

Разные корни a,b имеют размеры 24,12, ранги 2,1; сейчас largest=24. Сравните два отдельных DSU: union by size и union by rank (при равных рангах b под a). Какие НОВЫЕ корни, изменение ранга и largest верны? Оба хранят размер у нового корня.

A. size: a; rank: b; leave both ranks unchanged; largest=36 / size: a; rank: b; не менять ранги; largest=36

B. size: b; rank: a; leave both ranks unchanged; largest=36 / size: b; rank: a; не менять ранги; largest=36

C. size: a; rank: a; leave both ranks unchanged; largest=36 / size: a; rank: a; не менять ранги; largest=36

D. size: a; rank: a; leave both ranks unchanged; largest=24 / size: a; rank: a; не менять ранги; largest=24

4. DSU complexity

All unions are redundant after the first connected group. DSU uses path compression and union by size (example: n=12000, q=36011). Treat n and q as independent, without a fixed ratio. Which pair gives the sharpest standard general amortized time bound AMONG THESE OPTIONS and correctly counts total memory INCLUDING q output states?

После связной группы все union повторные. DSU использует сжатие путей и union by size (пример: n=12000, q=36011). Считайте n и q независимыми, без фиксированного соотношения. Какая пара даёт наиболее точную стандартную общую амортизированную оценку времени СРЕДИ ЭТИХ ОПЦИЙ и верно учитывает общую память, ВКЛЮЧАЯ q ответов?

A. Time O(n+q alpha(n)); memory O(n+q). / Время O(n+q alpha(n)); память O(n+q).

B. Time O(nq); memory O(n+q). / Время O(nq); память O(n+q).

C. Time O(n+q alpha(n)); memory O(n), including output. / Время O(n+q alpha(n)); память O(n), включая ответы.

D. Time O(n+q); memory O(1), including output. / Время O(n+q); память O(1), включая ответы.

5. Initialization and reset

One JSON-lines process has just answered DSU with every vertex merged. Its next request is a fresh DSU with n=10. BEFORE processing any operations, what internal [components, largest_size] values must initialization establish? This asks about internal state, not the returned states list.

Один процесс JSON-lines только что ответил на запрос DSU со всеми объединёнными вершинами. Следующий запрос - новый DSU с n=10. Какие внутренние [components, largest_size] должна установить инициализация ДО любых операций? Вопрос о внутреннем состоянии, не о возвращаемом списке states.

A. [10, 11]

B. [1, 30]

C. [10, 1]

D. [30, 0]

6. Kruskal trace

n=5; undirected [u,v,w] edges=[[0, 1, 12], [1, 2, -12], [0, 2, 17], [2, 3, 1], [2, 2, -90]]. What minimum spanning FOREST cost must sort-based Kruskal return?

n=5; неориентированные рёбра [u,v,w]=[[0, 1, 12], [1, 2, -12], [0, 2, 17], [2, 3, 1], [2, 2, -90]]. Какую стоимость минимального остовного ЛЕСА должен вернуть Краскал с сортировкой?

A. 1

B. -89

C. -72

D. 14

7. Ties and edge IDs

n=4; undirected edges with input IDs 0..3: [[0, 1, 7], [1, 2, 7], [0, 2, 7], [2, 2, -50]]. Which complete minimum spanning FOREST response must be accepted, although another optimum uses different IDs?

n=4; неориентированные рёбра с исходными ID 0..3: [[0, 1, 7], [1, 2, 7], [0, 2, 7], [2, 2, -50]]. Какой полный ответ минимального остовного ЛЕСА должен приниматься, хотя другой оптимум использует иные ID?

A. cost=14, edge_ids=[1, 1], components=2

B. cost=14, edge_ids=[1, 2], components=3

C. cost=14, edge_ids=[1, 2], components=2

D. cost=15, edge_ids=[1, 2], components=2

8. Spanning forest edge count

An undirected graph has n=17 vertices and c=4 connected components, counting isolated vertices. How many edge IDs must a spanning forest contain?

В неориентированном графе n=17 вершин и c=4 связных компонент, включая изолированные вершины. Сколько ID рёбер должен содержать остовный лес?

A. 13

B. 4

C. 16

D. 17

9. Benchmark medians and interpretation

Correctness was checked first. On the SAME seeded graphs/environment, both algorithms time only the method including result construction; compilation/parsing are excluded for both. Three runs in ms per size: n=100,m=250: K=[12, 21, 13], P=[5, 11, 4]; n=200,m=500: K=[14, 23, 15], P=[8, 14, 7]; n=300,m=750: K=[16, 25, 17], P=[11, 17, 10]. At size n=200, which K/P medians and conclusion are valid?

Корректность проверена заранее. ОДИНАКОВЫЕ графы с seed и среда; оба алгоритма измеряют только метод с построением результата, без компиляции/parsing. Три времени в мс: n=100,m=250: K=[12, 21, 13], P=[5, 11, 4]; n=200,m=500: K=[14, 23, 15], P=[8, 14, 7]; n=300,m=750: K=[16, 25, 17], P=[11, 17, 10]. При n=200 какие медианы K/P и вывод верны?

A. 14 / 7; compare the fastest single runs. / 14 / 7; сравнить самые быстрые одиночные запуски.

B. 52 / 29; compare the sums as medians. / 52 / 29; сравнить суммы как медианы.

C. 15 / 8; Prim; observations, not a proof of asymptotic complexity. / 15 / 8; Прим; наблюдения, не доказательство асимптотики.

D. 15 / 8; Kruskal; these three runs prove the complexity bound. / 15 / 8; Краскал; три запуска доказывают оценку сложности.

10. Prim frontier and stale edges

Lazy Prim has visited S=[0]; n=5, undirected input edges=[[0, 4, 10], [1, 4, -6], [0, 0, -2], [0, 1, 6]], IDs 0..3. Its heap contains all edges touching S, including stale entries. Which ID is accepted NEXT?

Ленивый Прим посетил S=[0]; n=5, неориентированные рёбра=[[0, 4, 10], [1, 4, -6], [0, 0, -2], [0, 1, 6]], ID 0..3. Heap содержит все рёбра, касающиеся S, включая устаревшие. Какой ID принимается СЛЕДУЮЩИМ?

A. 3

B. 2

C. 0

D. 1

11. Disconnected Prim

n=8, undirected edges=[[0, 1, 2]]. Prim starts an unvisited root after each exhausted heap, including isolated vertices. How many components must the forest response report?

n=8, неориентированные рёбра=[[0, 1, 2]]. После исчерпания heap Прим начинает с непосещённого корня, включая изоляты. Сколько компонент должен сообщать ответ-лес?

A. 1

B. 8

C. 7

D. 9

12. Safe edge and cut property

For n=4, cut S=[0, 2], edges [u,v,w]=[[0, 1, 12], [2, 3, 15], [0, 0, -20]]. The current forest extends to an optimum and has no edge crossing this cut. Why may Kruskal/Prim accept edge ID 0, even when another crossing edge ties it?

n=4, разрез S=[0, 2], рёбра [u,v,w]=[[0, 1, 12], [2, 3, 15], [0, 0, -20]]. Текущий лес продолжается до оптимума и не пересекает этот разрез. Почему Краскал/Прим может принять ID 0, даже при равном весе другого ребра разреза?

A. A minimum crossing edge is safe: exchange it into some optimum respecting the current forest. / Минимальное ребро разреза безопасно: обменом его можно включить в некоторый оптимум, содержащий текущий лес.

B. A negative self-loop is always safer than a crossing edge because it reduces total cost. / Отрицательный self-loop всегда безопаснее ребра разреза, поскольку снижает стоимость.

C. Every minimum crossing edge must belong to every optimum, including when weights tie. / Каждое минимальное ребро разреза обязано входить во все оптимумы, даже при равных весах.

D. Connecting two different vertices proves an edge is minimum, without comparing the cut. / Соединение разных вершин доказывает минимальность ребра без сравнения на разрезе.

13. Lazy Prim complexity

Every input pair may have parallel alternatives. Lazy-heap Prim uses adjacency lists, allows parallel edges and scans every component (example: n=5500, m=22010). Give a general worst-case bound for independent n and m, without assuming a fixed ratio. Which includes isolated vertices and heap entries?

У каждой пары могут быть параллельные рёбра. Прим с ленивой очередью использует списки смежности, допускает параллельные рёбра и обходит все компоненты (пример: n=5500, m=22010). Дайте общую худшую оценку для независимых n и m, без фиксированного соотношения. Какая учитывает изолированные вершины и элементы heap?

A. Time O(m log(m+1)); memory O(1), even with isolated vertices. / Время O(m log(m+1)); память O(1), даже с изолированными вершинами.

B. Time O(n+m); memory O(n), because the heap can hold at most n edges. / Время O(n+m); память O(n), ведь heap хранит не больше n рёбер.

C. Time O(n+m log(m+1)); memory O(n+m). / Время O(n+m log(m+1)); память O(n+m).

D. Time O(nm); memory O(nm), because an adjacency matrix is required. / Время O(nm); память O(nm), поскольку нужна матрица смежности.

14. SCC partition

n=6, directed edges=[[4, 5], [5, 4], [5, 0], [0, 1], [1, 0], [1, 2]]. Which partition is exactly the strongly connected components? Group order does not matter.

n=6, ориентированные рёбра=[[4, 5], [5, 4], [5, 0], [0, 1], [1, 0], [1, 2]]. Какое разбиение точно задаёт SCC? Порядок групп не важен.

A. [0, 1] | [2] | [3] | [4, 5]

B. [0, 1, 2, 3, 4, 5]

C. [0] | [1] | [2] | [3] | [4] | [5]

D. [0, 4, 5] | [1, 2, 3]

15. Equivalent SCC labels and condensation

Directed edges=[[0, 1], [1, 0], [0, 2], [1, 2], [2, 3], [2, 3], [3, 3]]. One correct SCC labeling is [20, 20, 40, 60]. Which response uses an EQUIVALENT labeling and its EXACT condensation? Labels may be renamed consistently.

Ориентированные рёбра=[[0, 1], [1, 0], [0, 2], [1, 2], [2, 3], [2, 3], [3, 3]]. Одно верное распределение меток SCC: [20, 20, 40, 60]. Какой ответ задаёт ЭКВИВАЛЕНТНЫЕ метки и их ТОЧНУЮ конденсацию? Метки можно согласованно переименовать.

A. labels=[80, 80, 80, 60], condensation=[[40, 60], [80, 40]]

B. labels=[80, 80, 40, 60], condensation=[[60, 40], [40, 80]]

C. labels=[80, 80, 40, 60], condensation=[[40, 60], [80, 40]]

D. labels=[80, 80, 40, 60], condensation=[[40, 60], [80, 40], [40, 60]]

16. Kosaraju pass order

Kosaraju's first DFS pass on the ORIGINAL graph appends vertices when they finish; finish=[2, 4, 3, 0, 1]. How should the second pass run?

Первый DFS Косарайю на ИСХОДНОМ графе добавляет вершины при завершении; finish=[2, 4, 3, 0, 1]. Как выполнить второй проход?

A. Traverse the transposed graph, starting unvisited roots in order [1, 0, 3, 4, 2]. / Обходить транспонированный граф, начиная непосещённые корни в порядке [1, 0, 3, 4, 2].

B. Traverse the original graph, starting roots in order [1, 0, 3, 4, 2]. / Обходить исходный граф, начиная корни в порядке [1, 0, 3, 4, 2].

C. Traverse the transposed graph, starting roots in order [2, 4, 3, 0, 1]. / Обходить транспонированный граф, начиная корни в порядке [2, 4, 3, 0, 1].

D. Sort vertices by numerical ID and use the original graph. / Отсортировать вершины по номерам и обходить исходный граф.

17. DFS finish trace

n=4, directed edges in adjacency order=[[2, 1], [2, 3], [1, 0]]. Outer DFS root order=[0, 1, 2, 3]; skip visited roots/neighbours. Which finish list is APPENDED by DFS (before reversal)?

n=4, ориентированные рёбра в порядке списков смежности=[[2, 1], [2, 3], [1, 0]]. Порядок корней внешнего DFS=[0, 1, 2, 3]; посещённых пропускать. Какой список finish ДОБАВЛЯЕТ DFS (до разворота)?

A. [2, 3, 1, 0]

B. [0, 1, 2, 3]

C. [0, 1, 3, 2]

D. [0, 2, 1, 3]

18. SCC time and memory

There may be many repeated cross-SCC edges. Kosaraju stores adjacency lists and their transpose, and deduplicates condensation with an expected-O(1) hash set (example: n=11000, m=33010). Give the standard general bounds for independent n and m, without assuming a fixed ratio.

Может быть много повторных рёбер между SCC. Косарайю хранит списки смежности и транспонированный граф, дедуплицирует конденсацию hash set с ожидаемым O(1) (пример: n=11000, m=33010). Дайте стандартные общие оценки для независимых n и m, без фиксированного соотношения.

A. Expected time O(n+m), memory O(n+m). / Ожидаемое время O(n+m), память O(n+m).

B. Time O(m log m), memory $O(n^2)$, because all edges must be sorted. / Время O(m log m), память $O(n^2)$, поскольку нужно сортировать все рёбра.

C. Time O(n+m), memory O(n), even including both adjacency lists and output. / Время O(n+m), память O(n), включая оба списка смежности и результат.

D. Time $O(n^2)$, memory O(1), because SCCs remove all edges. / Время $O(n^2)$, память O(1), поскольку SCC удаляют все рёбра.

19. DFS cycle colours

Why is GRAY the correct condition for detecting a DFS back edge?

Почему условие GRAY правильно обнаруживает обратное ребро DFS?

```
// WHITE=0, GRAY=1 (active), BLACK=2 (finished)
// Explore edge 22 -> 23
if (color[23] == 1) cycle = true;
```

A. A WHITE target already completed its DFS subtree. / WHITE означает, что поддерево цели уже завершено.

B. Only an edge from the source vertex can close a cycle. / Цикл может замкнуть только ребро из исходной вершины.

C. The target is still on the current DFS path; this edge closes a directed cycle. / Цель ещё находится на текущем пути DFS; ребро замыкает направленный цикл.

D. Any previously visited target proves a cycle, including BLACK targets. / Любая уже посещённая цель доказывает цикл, включая BLACK.

20. DFS reverse postorder trace

n=4, directed edges in adjacency order=[[3, 0], [3, 1], [0, 2]]. Outer DFS root order=[0, 1, 2, 3]; skip visited roots/neighbours. Which order does this DFS implementation return after REVERSING its finish list? Other topological orders may also exist.

n=4, ориентированные рёбра в порядке списков смежности=[[3, 0], [3, 1], [0, 2]]. Порядок корней внешнего DFS=[0, 1, 2, 3]; посещённых пропускать. Какой порядок вернёт эта реализация DFS после РАЗВОРОТА finish? Могут существовать и другие топологические порядки.

A. [3, 1, 0, 2]

B. [0, 2, 1, 3]

C. [2, 0, 1, 3]

D. [0, 1, 2, 3]

21. Cycles in the whole graph

For topo, n=7 and directed edges=[[0, 1], [2, 3], [3, 4], [4, 2]]. A DFS starting at 0 finishes without finding a cycle. What must the complete function return?

Для topo n=7, ориентированные рёбра=[[0, 1], [2, 3], [3, 4], [4, 2]]. DFS из 0 завершился без цикла. Что должна вернуть полная функция?

A. acyclic=true, order=[0,1]

B. acyclic=true, order=[0, 1, 2, 3, 4, 5, 6]

C. acyclic=false, order=[]

D. acyclic=false, order=[0,1]

22. Isolated and empty graphs

For topo, n=10 and edges=[]. Which response satisfies the assignment contract?

Для topo n=10, edges=[]. Какой ответ соответствует контракту задания?

A. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

B. acyclic=false, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

C. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1]

D. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 10]

23. Iterative DFS complexity

A star is also tested, so restarting every neighbour scan is expensive. DFS topological ordering must handle a chain of 90000 vertices without recursion-stack failure AND retain O(n+m) traversal time on arbitrary directed graphs. Which strategy guarantees both requirements?

Проверяется также звезда: начинать просмотр соседей заново дорого. DFS-топологическая сортировка должна обработать цепочку из 90000 вершин без переполнения стека вызовов И сохранять O(n+m) времени на произвольных ориентированных графах. Какой подход гарантирует оба требования?

A. Recursively visit the chain and assume every runtime permits that depth. / Рекурсивно посетить цепочку и считать такую глубину допустимой в любой среде.

B. At every stack step, rescan all neighbours from the beginning. / На каждом шаге стека просматривать всех соседей сначала.

C. Use explicit frames (vertex, next-neighbour index); finish on pop and scan each edge once. / Использовать явные кадры (вершина, индекс соседа); завершать при pop, каждое ребро смотреть один раз.

D. Push vertices in numeric order and return that order without DFS. / Положить вершины по номерам и вернуть этот порядок без DFS.

24. Why DAG relaxation works

DAG n=6, source=0, weighted directed edges=[[0, 1, 12], [1, 4, -17], [0, 4, 22]]; include all vertices, even isolates. Why does ONE topological-order relaxation pass suffice with negative weights, and what is distance[4] and total time with adjacency lists?

DAG n=6, source=0, взвешенные ориентированные рёбра=[[0, 1, 12], [1, 4, -17], [0, 4, 22]]; учитывайте все вершины, включая изоляты. Почему ОДНОГО прохода релаксаций в топологическом порядке достаточно при отрицательных весах; каковы distance[4] и полное время со списками смежности?

A. distance[4]=-5; predecessors are processed first; O(n+m). / distance[4]=-5; предшественники обработаны раньше; O(n+m).

B. distance[4]=-17; start every distance at zero; O(n+m). / distance[4]=-17; начать все расстояния с нуля; O(n+m).

C. distance[4]=-5; repeat every edge n times; O(nm). / distance[4]=-5; повторять все рёбра n раз; O(nm).

D. distance[4]=22; ignore negative edges; O(n+m). / distance[4]=22; игнорировать отрицательные рёбра; O(n+m).

25. Unreachable cycle

dag receives n=6, source=0, edges=[[0, 1, 13], [2, 2, -1]]. No vertex of the directed cycle is reachable from source. Which complete response is required?

dag получает n=6, source=0, рёбра=[[0, 1, 13], [2, 2, -1]]. Ни одна вершина направленного цикла не достижима из source. Какой полный ответ требуется?

A. acyclic=false, distance=[0], parent_edge=[-1]

B. acyclic=true, distance=[0, 13, null, null, null, null], parent_edge=[-1, 0, -1, -1, -1, -1]

C. acyclic=false, distance=[], parent_edge=[]

D. acyclic=true, distance=[], parent_edge=[]

26. Skipping unreachable vertices

The destination currently has null distance. Which rule prevents an unreachable vertex from creating a false shortest path?

Сейчас distance цели равен null. Какое правило не даст недостижимой вершине создать ложный кратчайший путь?

```
// distance[u] is null: u is unreachable
// edge u -> v has weight -20
Long du = distance[u];
// Select the correct relaxation rule.
```

A. If du == null, skip this edge; otherwise compute and compare du + weight. / Если du == null, пропустить ребро; иначе вычислить и сравнить du + weight.

B. Replace null with zero and relax normally. / Заменить null нулём и выполнить relax.

C. Set distance[v] to weight whenever the weight is negative. / При отрицательном весе присвоить distance[v] значение weight.

D. Replace null with a large sentinel and always add the negative weight. / Заменить null большим значением и всегда прибавлять отрицательный вес.

27. Tied paths and parallel edge IDs

DAG n=4, source=0; edges with input IDs 0..5: [[1, 2, -9], [2, 3, 2], [0, 1, 12], [0, 1, 12], [1, 2, -11], [0, 2, 1]]. Correct distance=[0, 12, 1, 3]. Which judgment of parent_edge witnesses is correct? No tie-break rule is required.

DAG n=4, source=0; рёбра с исходными ID 0..5: [[1, 2, -9], [2, 3, 2], [0, 1, 12], [0, 1, 12], [1, 2, -11], [0, 2, 1]]. Верный distance=[0, 12, 1, 3]. Какое суждение о parent_edge верно? Правило разрешения равенств не задано.

A. [-1, 3, 0, 1] is valid because any incoming edge is sufficient. / [-1, 3, 0, 1] верен: достаточно любого входящего ребра.

B. Only [-1, 2, 4, 1] is valid; parallel ties must use the first ID. / Только [-1, 2, 4, 1]: при равных параллельных рёбрах обязателен первый ID.

C. Both [-1, 2, 4, 1] and [-1, 3, 5, 1] are valid. / Оба массива [-1, 2, 4, 1] и [-1, 3, 5, 1] верны.

D. Only [-1, 3, 5, 1] is valid; direct paths must win every tie. / Только [-1, 3, 5, 1]: прямой путь обязан выигрывать каждое равенство.

28. Source and unreachable route

A DAG has n=6, source=2, only edge 2→3, and isolated vertex 0. Which pair of helper results is required: route(source), route(0)?

B DAG n=6, source=2, единственное ребро 2→3, вершина 0 изолирована. Какие результаты helper требуются: route(source), route(0)?

A. [2], []

B. [2], [0]

C. [], [0]

D. [], []

29. 64-bit arithmetic

What does this Java code PRINT? Check the numeric type at the moment of each addition/multiplication; assignment to long afterwards cannot undo int overflow.

Что НАПЕЧАТАЕТ этот Java-код? Проверьте числовой тип в момент каждого сложения/умножения: последующее присваивание в long не отменяет переполнения int.

```
int w=810000000;
long total=(long)w*5;
System.out.println(total);
```

A. -244967296

B. 810000000

C. 4050000000

D. -4050000000

30. Route reconstruction

n=7, source=4, target=0; directed [u,v,w] edges with input IDs=[[4, 1, 2], [1, 0, 1], [4, 0, 50]]; parent_edge=[1, 0, -1, -1, -1, -1, -1]. What does the called helper return, and what are its time/output-memory bounds for L route vertices with O(1) indexed edge lookup?

n=7, source=4, target=0; ориентированные рёбра [u,v,w] с исходными ID=[[4, 1, 2], [1, 0, 1], [4, 0, 50]]; parent_edge=[1, 0, -1, -1, -1, -1, -1]. Что вернёт вызванный helper и каковы время/память результата для L вершин маршрута при индексном доступе к ребру за O(1)?

A. [4, 1, 0]; O(L) time and O(L) returned-route memory / [4, 1, 0]; время O(L), память возвращаемого пути O(L)

B. [4, 1, 0]; O(1) time and memory / [4, 1, 0]; время и память O(1)

C. [4, 0]; O(n+m) time by searching the whole graph again / [4, 0]; время O(n+m), новый поиск по всему графу

D. [0, 1, 4]; O(L) time and O(L) returned-route memory / [0, 1, 4]; время O(L), память возвращаемого пути O(L)

Answers and explanations

Ответы и объяснения

Use the explanations to check your reasoning, not only the selected letter. / Сверяйте ход рассуждения, а не только букву.

1. C - DSU trace

The successive states are [[7, 2], [6, 3], [6, 3], [6, 3]]. Only merging distinct components decreases components; redundant/self unions leave the state unchanged. largest is the maximum component size, not the number of successful unions.

Последовательные состояния: [[7, 2], [6, 3], [6, 3], [6, 3]]. Только слияние разных компонент уменьшает components; повторный/self union ничего не меняет. largest - максимальный размер компоненты, не число успешных union.

2. A - Path compression state

Only the old path [14, 13, 12, 11] changes: every vertex on it points to root 10. Other entries and the root stay unchanged. Reading parent[u] after overwriting it would stop too soon.

Меняется только старый путь [14, 13, 12, 11]: каждая его вершина указывает на корень 10. Остальные элементы и корень прежние. Чтение parent[u] после перезаписи завершило бы обход слишком рано.

3. C - Size, rank and largest

Size chooses a by comparing 24 and 12. Rank chooses a; only equal ranks require an increment. The new root size is 36 under either heuristic; largest=max(24,36)=36. Rank is not component size.

Размер выбирает a, сравнив 24 и 12. Ранг выбирает a; увеличение нужно лишь при равенстве рангов. Размер нового корня в обоих случаях 36; largest=max(24,36)=36. Ранг не равен размеру компоненты.

4. A - DSU complexity

The two heuristics give inverse-Ackermann amortized operations. Parents/sizes need O(n); q returned states need O(q).

Обе эвристики дают амортизированную оценку с обратной функцией Аккермана. Массивы требуют O(n), возвращаемые состояния O(q).

5. C - Initialization and reset

Each request initializes its own state, regardless of the preceding task. The empty DSU has [0,0]; otherwise there are n singleton components and largest=1.

Каждый запрос инициализирует собственное состояние независимо от предыдущей задачи. Пустой DSU имеет [0,0]; иначе n одиночных компонент и largest=1.

6. A - Kruskal trace

The optimum is 1. Ignore self-loops, accept a sorted edge only when its endpoints have DIFFERENT DSU representatives, and cover every input component. Isolated vertices add zero cost.

Оптимум 1. Self-loop исключаются; сортированное ребро принимается только при РАЗНЫХ представителях DSU концов; охватываются все исходные компоненты. Изоляты дают нулевую стоимость.

7. C - Ties and edge IDs

Independent forest enumeration gives minimum cost 14 and 2 components. IDs [1, 2] are distinct, acyclic and cover every original component. Other tied optima may choose different IDs; duplicate IDs, wrong totals or wrong components are invalid.

Перебор лесов даёт минимальную стоимость 14 и 2 компонент. ID [1, 2] различны, ацикличны и охватывают все исходные компоненты. Другие равнооптимальные ID допустимы; повторы, неверная сумма или число компонент - нет.

8. A - Spanning forest edge count

A tree on k vertices has k-1 edges. Summing across c components gives n-c, including zero edges for singleton components.

В дереве из k вершин k-1 рёбер. Сумма по c компонентам равна n-c; одиночная вершина даёт ноль рёбер.

9. C - Benchmark medians and interpretation

Sort each triple and take its middle value: 15 and 8. These data compare this reproducible experiment only. Kruskal still has O(n+m log(m+1)) time; repeated measurements do not prove an asymptotic bound.

После сортировки троек берутся средние элементы: 15 и 8. Данные сравнивают этот воспроизводимый опыт. Время Краскала O(n+m log(m+1)); повторы не доказывают асимптотику.

10. A - Prim frontier and stale edges

Only edges with exactly one endpoint in S cross the frontier. Edge ID 3 is the lightest such edge. Stale internal edges are discarded; an edge with neither endpoint visited is not a frontier candidate even if its weight is lower.

Границу пересекают только рёбра ровно с одним концом в S. ID 3 - самое лёгкое из них. Внутренние устаревшие рёбра пропускаются; ребро с обоими непосещёнными концами не кандидат границы, даже при меньшем весе.

11. C - Disconnected Prim

The original graph has 7 connected components. A Prim restart counts one whole component; isolated vertices each count once. Loops/parallel edges do not merge additional components.

В исходном графе 7 компонент связности. Каждый перезапуск Прима учитывает целую компоненту; каждый изолят считается один раз. Петли/параллельные рёбра не объединяют дополнительные компоненты.

12. A - Safe edge and cut property

The cut respects the current forest. Add a light crossing edge to an extending optimum; if a cycle appears, remove another crossing edge of no smaller weight. This proves existence of an optimum containing it, not membership in every tied optimum.

Разрез согласован с текущим лесом. Добавив лёгкое ребро в продолжающий его оптимум, при цикле удаляют другое ребро разреза не меньшего веса. Доказывается существование оптимума с этим ребром, а не принадлежность всем оптимумам при равенстве весов.

13. C - Lazy Prim complexity

A lazy heap can hold O(m) edge candidates. Heap operations add log(m+1), while initializing/scanning vertices adds O(n). Adjacency lists require O(n+m).

В ленивом heap возможно O(m) кандидатов. Операции очереди добавляют log(m+1), инициализация и проход по вершинам O(n); списки смежности занимают O(n+m).

14. A - SCC partition

Two vertices share an SCC exactly when each can reach the other. Applying that test gives [0, 1] | [2] | [3] | [4, 5]. A one-way path alone is insufficient; isolated and noncyclic singletons remain separate.

Две вершины в одной SCC тогда и только тогда, когда достижимы друг из друга. Проверка даёт [0, 1] | [2] | [3] | [4, 5]. Одностороннего пути недостаточно; изоляты и нециклические одиночные вершины отдельны.

15. C - Equivalent SCC labels and condensation

SCC numbers are names. Equal labels must describe exactly mutual-reachability groups. Condensation uses the chosen labels, removes intra-SCC edges and contains each directed pair once; it cannot retain duplicates or reverse directions.

Номера SCC - имена. Совпадение меток должно точно задавать группы взаимной достижимости. Конденсация использует выбранные метки, исключает внутренние рёбра и содержит каждую направленную пару один раз; повторы и обращение направлений неверны.

16. A - Kosaraju pass order

With the first pass on G, the second pass uses G transpose and decreasing finishing time. Each new DFS root identifies one SCC.

После первого прохода на G второй использует транспонированный граф и убывающий порядок завершения. Каждый новый корень DFS задаёт одну SCC.

17. C - DFS finish trace

The actual discovery list is [0, 1, 2, 3]; finishing occurs only after all neighbours, giving [0, 1, 3, 2]. Reversal gives [2, 3, 1, 0]. A BLACK neighbour is skipped rather than restarted.

Фактический список обнаружения [0, 1, 2, 3]; завершение только после всех соседей даёт [0, 1, 3, 2]. Разворот даёт [2, 3, 1, 0]. BLACK-соседа пропускают, не запускают повторно.

18. A - SCC time and memory

Two DFS passes and edge scanning are linear. The graph, transpose, traversal state and up to O(m) condensation pairs fit O(n+m) memory; hashing makes deduplication expected linear time.

Два DFS и проход по рёбрам линейны. Граф, транспонированный граф, состояние обхода и до O(m) пар занимают O(n+m); hash set даёт ожидаемую линейную дедупликацию.

19. C - DFS cycle colours

GRAY identifies an ancestor on the active call/frame stack. A BLACK target may be a harmless forward/cross edge and is not by itself proof of a cycle.

GRAY обозначает предка на активном стеке. BLACK может быть целью безопасного прямого/поперечного ребра и само по себе цикл не доказывает.

20. A - DFS reverse postorder trace

The actual discovery list is [0, 2, 1, 3]; finishing occurs only after all neighbours, giving [2, 0, 1, 3]. Reversal gives [3, 1, 0, 2]. A BLACK neighbour is skipped rather than restarted.

Фактический список обнаружения [0, 2, 1, 3]; завершение только после всех соседей даёт [2, 0, 1, 3]. Разворот даёт [3, 1, 0, 2]. BLACK-соседа пропускают, не запускают повторно.

21. C - Cycles in the whole graph

Topo must inspect every component. A directed cycle or self-loop elsewhere makes a topological ordering impossible; the contract requires an empty order when cyclic.

Topo проверяет все компоненты. Цикл или self-loop в другой области исключает топологический порядок; при цикле контракт требует пустой order.

22. A - Isolated and empty graphs

An edgeless graph is acyclic; every vertex must appear exactly once. Any permutation works, including the empty list when n=0.

Граф без рёбер ацикличен; каждая вершина появляется ровно один раз. Подходит любая перестановка, при n=0 пустой список.

23. C - Iterative DFS complexity

Frames replace recursive state and remember progress. They take O(n) traversal memory; together with adjacency lists the implementation uses O(n+m) time/memory.

Кадры заменяют рекурсивное состояние и запоминают прогресс. Сам стек занимает O(n); со списками смежности время и память O(n+m).

24. A - Why DAG relaxation works

In topological order every edge enters a later vertex. When u is processed, all reachable predecessor paths have already been considered, so its distance is final. The shortest target distance is -5; unreachable tails are skipped. Topo, initialization and one relaxation per edge take O(n+m).

В топологическом порядке каждое ребро ведёт в более позднюю вершину. При обработке u все пути от достижимых предшественников уже рассмотрены, поэтому расстояние окончательно. Кратчайшее расстояние до цели -5; недостижимые начала рёбер пропускаются. Topo, инициализация и по одной relax на ребро требуют O(n+m).

25. C - Unreachable cycle

Acyclicity is checked over the whole input, including unreachable components. A self-loop is also a cycle. On any cycle the contract returns acyclic=false with both arrays empty.

Ацикличность проверяется во всём входе, включая недостижимые компоненты. Self-loop также цикл. При любом цикле контракт возвращает acyclic=false и оба пустых массива.

26. A - Skipping unreachable vertices

No route reaches u, so no route through this edge is known. Relaxing a null/infinite distance can create fake reachability or overflow.

Путь до u отсутствует, поэтому через это ребро путь не возникает. Relax от null/бесконечности может дать ложную достижимость или переполнение.

27. C - Tied paths and parallel edge IDs

IDs 2 and 3 are equal parallel edges into vertex 1. Vertex 2 may use edge 4 via 1 or tied direct edge 5. Edge 0 enters 2 but gives a larger cost. Other reachable vertices keep their valid parents in both witnesses.

ID 2 и 3 - равные параллельные рёбра в вершину 1. Для 2 допустимо ребро 4 через 1 или равнооптимальное прямое 5. Ребро 0 входит в 2, но даёт большую стоимость. Остальные достижимые вершины сохраняют верных предков в обоих ответах.

28. A - Source and unreachable route

The source has distance zero and parent_edge -1, so its route is [source]. The unreachable vertex has null distance and parent_edge -1, so its route is [].

Источник имеет расстояние 0 и parent_edge=-1, его маршрут [source]. Недостижимая вершина имеет distance=null и parent_edge=-1, её маршрут [].

29. C - 64-bit arithmetic

The printed value is 4050000000. The mathematical weighted sum is 4050000000. The accumulation/multiplication already has a long operand, so it preserves the full 64-bit sum.

Напечатано 4050000000. Математическая взвешенная сумма 4050000000. Накопление/умножение уже имеет long-операнд, поэтому сохраняет полную 64-битную сумму.

30. A - Route reconstruction

Follow the parent edge of 0 back to its tail until source, then reverse once: [4, 1, 0]. Edge IDs are not vertex IDs. One step per route edge plus reversal/output gives O(L) time and O(L) returned-list memory; no new graph search is needed.

От 0 следовать по parent_edge к началу ребра до source, затем один раз развернуть: [4, 1, 0]. ID рёбер не равны номерам вершин. Один шаг на ребро пути, разворот и вывод дают O(L) времени и O(L) памяти списка; новый поиск по графу не нужен.